

The Kibana Dashboard

This note explains how our live dashboard works and how to build it again if we ever need to. The dashboard is not saved in this repo because it lives inside Kibana on the T-Pot VM. So if that VM is ever wiped and set up again, the dashboard is gone, and this file is how we bring it back.

The main idea

The first thing to get clear: Kibana does not run our model. Our Python script does. The live classifier (03-scripts/live_runs.py) is the part that loads the XGBoost model, watches the honeypot, and decides what each attack is. Kibana only reads and draws data from an Elasticsearch index. So "showing our predictions in Kibana" really just means this: Python writes each prediction into an index, and Kibana draws charts from that index.

Here is the whole flow:

live_runs.py sees an attack -> decides what it is (from XGBoost)

-> writes one small JSON record into the Elasticsearch index "ml-predictions"

-> the Kibana dashboard reads that index and draws the charts

The Python side is already finished inside live_runs.py. This file is mostly about the Kibana side — the index we write to, and the charts we built on top of it.

The index we write to

Every prediction goes into one index called **ml-prediction**, and we told ES exactly what type each field is instead of letting it guess, because the charts only work if the time is a real date and the class is a keyword. The fields are:

@timestamp date (when the attack finished — the time line for every chart)

predicted_class keyword (scan / online_guessing / post_exploitation / web_attack / denial_of_service)

confidence float (saved as a number between 0 and 1)

event_count integer (how many events the attack made)

duration_sec float (how long the attack lasted)

run_start date

run_end date

probs.<class> float (the score for each of the five classes)

One choice worth keeping in mind: we save the confidence and all the scores as numbers between 0 and 1, not as percentages. Kibana turns them into percentages when it shows them (the "Percent" format).

Each attack we classify becomes one record — one saved line that stays there and can be searched later.

Building the dashboard again

Everything below is done in the Kibana website at <https://192.168.32.138:64297>. The SSH tunnel to Elasticsearch has to be open, the same as when we run the classifier.

Step 1 — tell Kibana the index is there (the data view)

Kibana will not draw charts from an index it doesn't know about, so first we register a data view:

Go to Stack Management -> Data Views -> Create data view. Give it the name ml-predictions, set the index pattern to ml-predictions* (the star so it still matches if the index is ever split into more), and pick @timestamp as the time field. Save it.

Before building anything, a quick check: open Discover, choose the ml-predictions data view, set the time to Last 24 hours, and make sure our prediction records show up as rows. If they show, the rest will work. One small thing to know: our records are saved in UTC time, but Kibana shows our local time, so the times look a few hours off.

Step 2 — the four charts

Make a new dashboard, and for a live demo turn on auto-refresh (every 60 seconds) so it updates on its own while we attack. Then we add four charts, all reading the ml-predictions data view:

1. Attack Types Over Time — a stacked bar chart. The bottom axis is @timestamp as a date histogram, the side axis is the count of records, split by predicted_class . This is the main one: every attack we launch adds a colored bar in its own color.

2. Detections by Class — a pie chart, split by predicted_class, sized by count. A simple view of how many of each attack type we have seen.

3. Avg of Confidence — one big number, the average of the confidence field, shown as a percent. This is a nice number to point at. It is also honest — it sits a bit below 99% because the hard web/DoS runs come in with lower confidence, which is exactly the behaviour we wrote about in our error analysis.

4. Recent Detections — a table, newest on top, showing predicted_class, @timestamp, confidence, event_count, and duration_sec, sorted by @timestamp going down. This is the plain log of what we detected.

Running the demo

With the dashboard open and auto-refresh on, we run the classifier in one terminal and fire attacks from another kali terminal:

```
python3 live_runs.py
```

Each result writes a record, and after about 60 seconds the charts update — the timeline grows a bar, the pie gains a slice, the average-confidence number settles, and a new row shows up at the top of the table.

The attacks in another terminal:

```
nmap -T4 -F 192.168.32.138    # scan — wait for the verdict + return to idle
```

```
hydra -t 4 -l root -P /tmp/pw.txt ssh://192.168.32.138 # online_guessing (pw: letmein is in the list)
```

```
sshpass -p letmein ssh -o StrictHostKeyChecking=no root@192.168.32.138 "id; uname -a; ls -la /; cat /etc/passwd; ps aux"
```

```
nikto -h http://192.168.32.138 # web_attack
```

```
sudo hping3 -S -p 80 -i u2000 -c 5000 192.168.32.138
```